

Short Questions 4

Josh Manchester
Department of Computer Science
University of Colorado Colorado Springs
Colorado Springs, CO 80918
jmanches@uccs.edu

Abstract

This paper addresses three topics in Monte Carlo reinforcement learning: how agents can explore without relying on Exploring Starts, the distinction between on-policy and off-policy methods, and the importance sampling ratio used to transfer returns between policies. These concepts build directly on the Exploring Starts limitations discussed in Short Questions 3, showing how practical RL agents handle exploration and learn from experience generated under different policies.

Question 1

Describe how an RL agent, one that is not forced to randomly perform every $\langle \text{state}, \text{action} \rangle$ pair at the beginning of an episode, can explore every $\langle \text{state}, \text{action} \rangle$ pair during training. How can we write a non-Exploring-Starts algorithm that learns values? Write out any formulas needed.

The solution is to use an ε -soft policy, where the agent balances exploitation and exploration at every step (Sutton and Barto 2018). Exploitation means doing the best action all the time—picking the current best action in every state. Exploration means not doing that all the time—trying non-best actions sometimes. Instead of forcing exploration at the start of each episode, the agent explores *throughout* the episode by occasionally taking random actions.

The most common version is the ε -greedy policy. Given a small $\varepsilon > 0$ and $|\mathcal{A}(s)|$ possible actions in state s , every action gets at least probability $\varepsilon/|\mathcal{A}(s)|$:

$$\pi(a | s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|\mathcal{A}(s)|} & \text{if } a = a^* = \arg \max_{a'} Q(s, a') \\ \frac{\varepsilon}{|\mathcal{A}(s)|} & \text{if } a \neq a^* \end{cases}$$

As a concrete example, suppose $\varepsilon = 0.10$ and $|\mathcal{A}(s)| = 5$. Every action is possible with probability at least $0.10/5 = 0.02$. The best action a^* gets probability $1 - 0.10 + 0.10/5 = 0.92$. We can verify: $0.92 + 4(0.02) = 1.0$. So the agent picks the best action 92% of the time but still tries every other action 2% of the time. No state-action pair is ever completely ignored.

Copyright © 2024, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

The full algorithm is **On-Policy First-Visit MC Control** (Sutton & Barto, p. 101). We initialize $Q(s, a)$ arbitrarily and set π to be ε -greedy with respect to Q . For each episode, we generate the episode using π , then work backward through the steps computing $G \leftarrow \gamma G + R_{t+1}$. For the first visit to each (S_t, A_t) pair, we append G to its return list and update $Q(S_t, A_t) \leftarrow \text{average}(\text{Returns}(S_t, A_t))$. Then we find $A^* \leftarrow \arg \max_a Q(S_t, a)$ and update the policy to be ε -greedy with respect to the new Q values. This cycle of evaluation and improvement repeats, converging toward the optimal ε -soft policy.

This works because the ε -soft guarantee means every action in every reachable state has a nonzero probability of being selected at every time step. The agent does not need special starting conditions to explore—it explores naturally as a side effect of the policy itself.

Question 2

Differentiate between on-policy and off-policy reinforcement learning algorithms.

The difference comes down to whether the agent learns about the same policy it is using to act, or a different one (Sutton and Barto 2018).

On-policy methods start with a policy π and improve it repeatedly toward π_* . The agent follows π , collects experience, and uses that experience to improve the same π . The ε -greedy MC control algorithm from Question 1 is a good example—the agent acts ε -greedy, evaluates the ε -greedy policy, and improves it. The tradeoff is that the learned policy can never be fully greedy because it must keep exploring, so on-policy methods converge to the best ε -soft policy rather than the true optimal policy.

Off-policy methods use two separate policies: a **behavior policy** b that generates the episodes and a **target policy** π that the agent is trying to learn about. The agent is acting using b , but trying to learn π . One way to think about it is that someone else (or the agent itself) uses b well, generating lots of episodes of training, and the agent learns π from that experience. The behavior policy can be ε -greedy or even fully random, while the target policy can be purely greedy. This separation means the agent can learn the optimal policy while still exploring freely.

Off-policy learning is the more general approach. On-

policy learning is actually the special case where $b = \pi$ —the behavior and target policies are the same (Sutton and Barto 2018). However, off-policy methods come with higher variance and may need another environment or an expert to learn from, which can be costly. They also require a way to correct for the mismatch between the two policies, which is where importance sampling comes in.

Question 3

When an agent learns $v_\pi(s)$ in a target policy π from a behavior policy b , it needs to transfer G_t values (Returns) from policy b to policy π . What formula can an agent use to do so? Write the formula and discuss it briefly.

The core idea is that we can take values from one distribution and scale them by a weighted factor to estimate another distribution. As an analogy, suppose we know the rainfall distribution in downtown Colorado Springs but want to estimate the rainfall in north Colorado Springs. If we have a way to relate the two distributions, we can scale the values we already have instead of collecting new data from scratch.

In off-policy RL, the behavior policy b generates episodes with discounted returns $G_{t,b}$ at each time step. We want to transfer these into returns under the target policy: $G_{t,\pi} = \rho_t \cdot G_{t,b}$. The scaling factor ρ_t is the **importance sampling ratio** (Sutton and Barto 2018). It is the probability of the states and actions occurring under the target policy π from time t to the end of the episode, divided by the probability under the behavior policy b :

$$\rho_t = \frac{\pi(a_t | S_t) \cdot \pi(a_{t+1} | S_{t+1}) \cdot \dots \cdot \pi(a_{T-1} | S_{T-1})}{b(a_t | S_t) \cdot b(a_{t+1} | S_{t+1}) \cdot \dots \cdot b(a_{T-1} | S_{T-1})} = \prod_{k=t}^{T-1} \frac{\pi(A_k | S_k)}{b(A_k | S_k)}$$

The state transition probabilities $p(s' | s, a)$ appear in both the numerator and denominator and cancel out, so the ratio depends only on the two policies and the action sequence—not on the MDP dynamics.

With ρ_t in hand, we can compute $V_\pi(s) = \mathbb{E}[\rho_t \cdot G_{t,b} | S_t = s]$. The book gives two ways to do this. Define $\tau(s)$ as the set of time points at which state s occurs in episodes of training using behavior policy b , and $|\tau(s)|$ as the number of such visits.

Ordinary importance sampling divides the sum of weighted returns by the number of visits:

$$V_\pi(s) = \frac{\sum_{t \in \tau(s)} \rho_t G_{t,b}}{|\tau(s)|}$$

This estimator is unbiased but can have high variance because ρ_t is a product over many time steps—if the episode is long, the product can become very large or very small.

Weighted importance sampling divides by the sum of the ratios instead:

$$V_\pi(s) = \frac{\sum_{t \in \tau(s)} \rho_t G_{t,b}}{\sum_{t \in \tau(s)} \rho_t}$$

This version is biased but has much lower variance in practice, which usually makes it the better choice (Sutton and Barto 2018). The requirement for importance sampling to work is **coverage**: $b(a | s) > 0$ wherever $\pi(a | s) > 0$, so that the denominator is never zero.

AI Use Statement

Claude (Anthropic) was used to format my answers in AAAI LaTeX format.

References

Sutton, R. S.; and Barto, A. G. 2018. *Reinforcement Learning: An Introduction*. Cambridge, MA: MIT Press, 2nd edition. Available free at <http://incompleteideas.net/book/the-book-2nd.html>.